Design and Implementation of an 8-bit RISC Processor: From RTL to Gate-Level Layout

Victor Lu
Dept. of Biomedical Engineering
Johns Hopkins University
Baltimore, United States
wlu33@jh.edu

***Abstract*—This paper presents the complete design flow—including simulation, synthesis, and physical implementation—of an 8-bit Reduced Instruction Set Computer (RISC). The goal is to implement a RISC processor architecture with a datapath, a Finite State Machine (FSM) controller, and a memory module. The RISC processor is designed using Verilog and it's modularized, integrating an Arithmetic Logic Unit (ALU), four general-purpose registers, and a dedicated control unit to achieve fetch, decode, and execute cycles.**

The processor supports instruction sets, which include arithmetic operations (ADD, SUB), logical operations (AND, NOT), memory access (RD, WR), and branching (BR, BRZ). Functional verification was performed on testbench by preloading a hex-based assembly program into memory to validate all instruction types. Simulation confirms the functionality of the design.

Following behavioral verification, the design was synthesized using the Genus tool. Post-synthesis verification was conducted to validate the gate-level netlist, which ensures functional equivalence after logic optimization. Finally, the design proceeded to Place and Route(PNR) stage for physical implementation. This stage involved floorplanning, standard cell placement, Clock Tree Synthesis (CTS), and routing. The successful completion of these stages confirms the architecture's viability for physical realization.

***Index Terms*—RISC Processor, Verilog, Logic Synthesis, Finite State Machine (FSM), Datapath, Post-Synthesis Verification, Cadence**

I. Introduction

The development of efficient and specialized processors lies at the center of modern digital system design. Reduced Instruction Set Computers (RISC) have become a popular architecture for embedded systems due to their simplified instruction sets, which allow for streamlined hardware implementation, faster clock cycles, and efficient pipelining [1]. This project focuses on the design, verification, and physical implementation of an 8-bit RISC Stored-Program Machine (SPM), demonstrating the full Application-Specific Integrated Circuit (ASIC) design flow from Register Transfer Level (RTL) coding to physical layout.

The primary objective of this work is to build a compact yet functional processor capable of executing general-purpose tasks. The design emphasizes modularity, separating the control logic (FSM) from the datapath and memory, which simplifies verification and potential future expansion [2].

This paper details the complete design workflow. Section II describes the system's architecture, including datapath components and the FSM controller. Section III outlines the verification methodology, highlighting the use of self-checking testbenches. Section IV discusses the synthesis results using the Genus technology node. Section V shows the Place and Route(PNR) results. Section VI displays the effects of timing constraints on various parameters of the chip. Finally, Section VII concludes the paper with a summary of the project's achievements.

II. Architecture

A. Datapath

The datapath facilitates data movement and processing. It consists of a set of 8-bit general-purpose registers (R0–R3) for temporary data storage, a Program Counter (PC) for instruction tracking, and specialized registers such as the Instruction Register (IR) and Address Register (Add_R) for memory interface operations. The datapath contains an ALU that performs essential operations on incoming signals, including ADD, SUBTRACT, bit-wise AND, and NOT. Data flow is controlled via two primary internal buses, Bus 1 and Bus 2, with multiplexers selecting inputs from registers, memory, or ALU outputs based on control signals.

B. Control Unit (FSM)

The control unit is implemented using a Finite State Machine (FSM) that orchestrates datapath operations. As illustrated in the state transition diagram (Fig. 1), the FSM operates through three primary phases: Instruction Fetch, Decode, and Execute.

The machine initializes in the S_idle state. Upon the de-assertion of the global reset signal ($reset = 0$), the FSM transitions to S_Fetch1, initiating the instruction cycle. The fetch phase spans two states, S_Fetch1 and S_Fetch2, designed to accommodate memory access timing. During the transition from S_Fetch2 to the decode stage, the Program Counter (PC) is updated, and the current instruction is latched into the Instruction Register (IR).

The S_decode state serves as the central decision node. Based on the opcode decoded from the IR, the FSM branches into one of five distinct execution paths:

- **Arithmetic/Logic:** Transitions to $S_execution$ (ALU), where computations are performed.

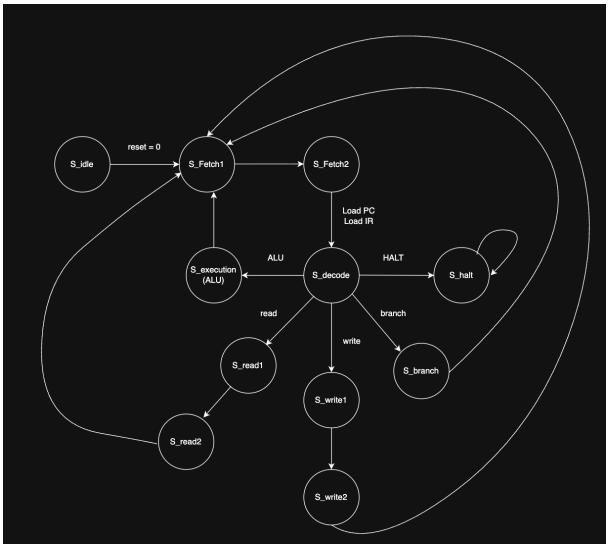


Fig. 1. The FSM State Transition Diagram.

- **Memory Read:** Initiates a sequence (S_read1, S_read2) to handle address setup and data latching.
- **Memory Write:** Triggers (S_write1, S_write2) to ensure stable data writing.
- **Control Flow:** Directs the FSM to S_branch, where the PC is updated if the branch condition is met.
- **Termination:** Transitions to S_halt, where the processor loops indefinitely.

Upon completion of the execution phase, the FSM loops back to S_Fetch1.

C. ALU and Memory

The ALU unit performs arithmetic operations (ADD, SUB) and logical operations (AND, NOT). The memory module contains 2048 D-flip-flops, organized as 256 memory locations of 8 bits each.

III. Behavioral Simulation

To evaluate the design, the following operations were tested: ADD, SUB, AND, NOT, and Branch-on-Zero. Fig. 2 displays the waveform for the ADD operation. The Opcode for Addition is '0001', visible in the four Most Significant Bits (MSB) of the IR signal at 145 ns. The remaining bits define the operands; '0100' indicates adding R1 and R0, storing the result in R0. This is confirmed by the change in the R0 signal. To test the memory datapath, the result in R0 is written to address 224, reflected at 205 ns in the 'd_to_mem' and 'addr' signals.

Branching logic was verified by performing an addition on R2 (value 0) with itself. As the result is zero, the zero flag is raised (Fig. 3). This triggers the branching mechanism, skipping the immediate next operation (a trap state at address 226) and correctly proceeding to write R0 into memory location 227.

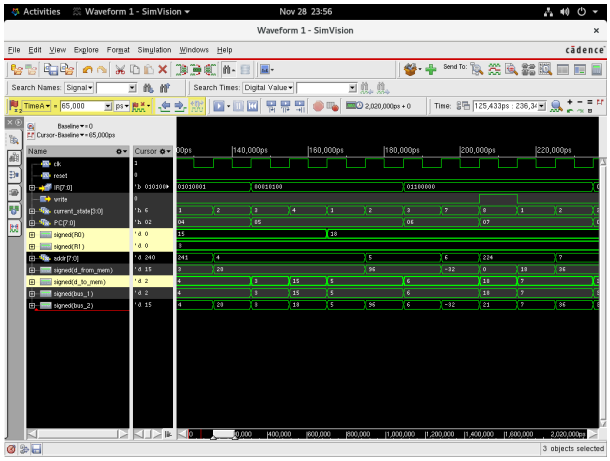


Fig. 2. Pre-synthesis simulation: Addition.

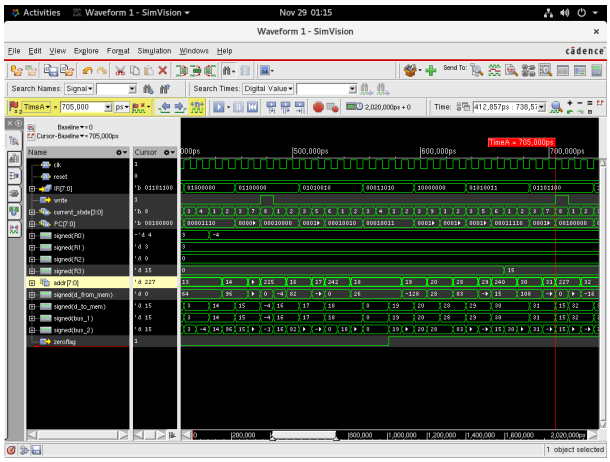


Fig. 3. Pre-synthesis simulation: Branching.

IV. Synthesis

A. Physical Information

Synthesis was performed with the Genus tool. Results align with expectations. As shown in Fig. 4, the design utilizes 2048 DFFQXL cells, corresponding to the memory array. The core logic utilizes 69 D-Flip-Flops (DFFs), which accounts for eight 8-bit registers including general-purpose registers (R0-R3), PC, IR, address and reg_y, a 4-bit register current_state, and a 1-bit register reg_z that holds the zero-flag.

Figures 5 and 6 illustrate the breakdown of total area and power consumption by instance type. Sequential cells occupy the most area because the design's storage requirements are synthesized into a sheer number of DFFs. ALUs perform combinatorial operations and are a part of the logic instance type; these operations require dedicated circuits and therefore contribute significantly to the logic area. Notably, clock-gating cells occupy a small area (9.2%) while still consuming a significant portion of the overall internal power(45%). This can be explained by the high dynamic power caused by the continuous switching behavior of the clocks, as well as the fact

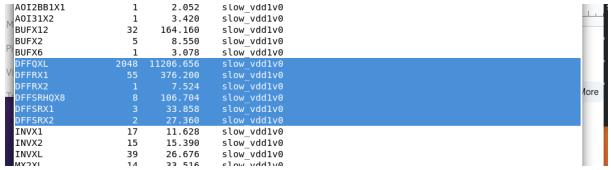


Fig. 4. Synthesis: DFF count.

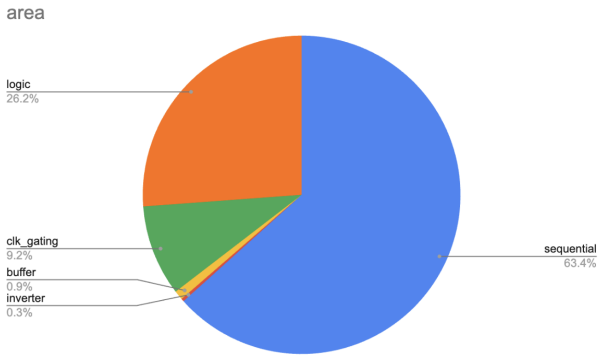


Fig. 5. Synthesis: chip area pie chart.

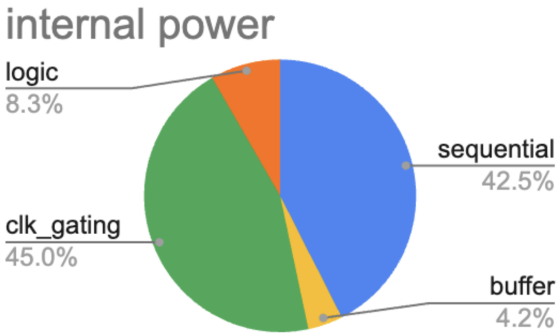


Fig. 6. Synthesis: internal power pie chart.

that clock gating strategies may not yet be fully optimized.

To provide a better physical view of the synthesis result, a schematics is generated and shown in Fig. 8. As expected, there are three main synthesized modules: a controller, a memory module and a processor. It's worth noticing that these modules are not sized proportional to their actual dimensions: the size of the synthesis result is determined by the amount of inputs to and outputs from the module.

B. Post-Synthesis Simulation

A summary of the synthesis netlist is provided in Fig. 9. The total instance count rises from 5140 to 5853. This increase occurs because the synthesis report counts only explicitly instantiated RTL modules, while the post-synthesis simulation utilizes the fully elaborated gate-level netlist, which includes optimization logic and primitives inserted by the synthesizer.

To verify the gate-level netlist, an ADD operation was simulated (Fig. 10). The waveform confirms that R_0 and R_1

```
                  Capture       Launch
     Clock Edge:+   10000            0
     Src Latency:+      0            0
     Net Latency:+      0 (I)        0 (I)
         Arrival:=  10000            0

   Output Delay:-    2000
  Required Time:=    8000
   Launch Clock:-       0
     Data Path:-    1682
         Slack:=    6318
```

Fig. 7. Synthesis: Slack time.

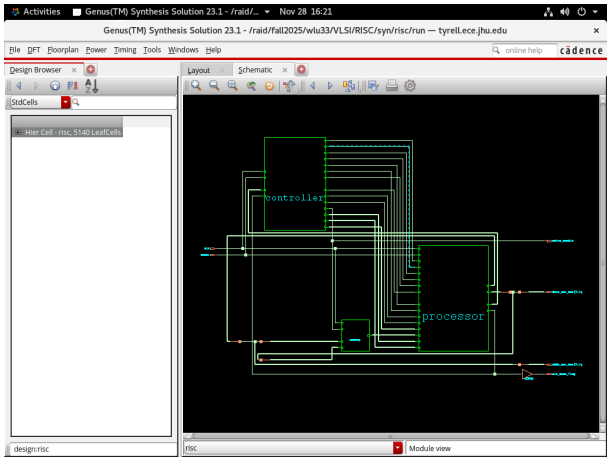


Fig. 8. Synthesis: Schematic overview.

```
Building instance specific data structures.
Loading native compiled code:     ..................... Done
Design hierarchy summary:
                         Instances  Unique
        Modules:             5,853     759
        UDPs:                4,757      14
        Primitives:         11,508       6
        Timing outputs:      5,730     432
        Registers:           2,533     173
        Scalar wires:       10,812       -
        Always blocks:           1       1
        Initial blocks:          3       3
        Pseudo assignments:      5       -
        Timing checks:      17,469       -
        Delayed tcheck signals:  5,549   1,367
        Simulation timescale:    1ps
Writing initial simulation snapshot: worklib.ACHCONX2:v
```

Fig. 9. Post-synthesis: Summary.

are successfully added, stored in R_0, and the result is written to memory address 224, matching the behavioral simulation.

V. Place and Route

A. Physical Layout

Following post-synthesis verification, Place and Route (PNR) was performed. This flow includes floorplanning, placement, Clock Tree Synthesis (CTS), and routing.

Fig. 11 is the final layout with only signal lines active in the field of view. Fig. 12 illustrates the floorplan: the memory array occupies the majority of the area, followed by the processor logic. Table I summarizes the physical parameters. The instance count and area increased significantly due to

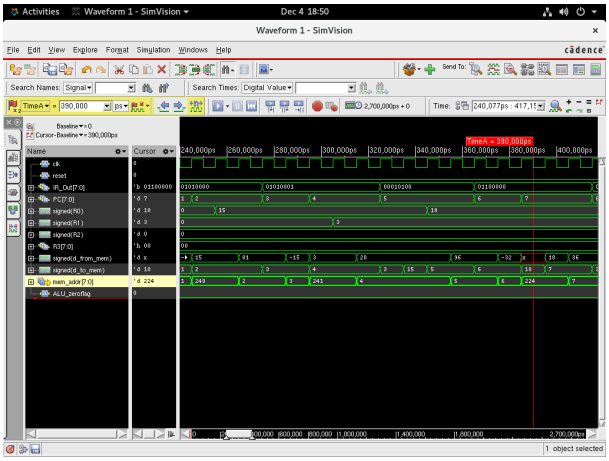


Fig. 10. Post-synthesis: ADD simulation.

TABLE I
SUMMARY OF CHIP PROPERTIES

	Chip Properties			
	Total Instances	***Area*** (μm^2)	***Density***	***WNS (ns)***
Values	13464	41122.76	50.25%	5.906

the insertion of buffers and inverters during CTS and hold-time fixing. The Worst Negative Slack (WNS) remains positive ($5.906ns$), indicating successful timing closure.

Fig. 13 and Fig. 14 display the routing of selected metal layers. The layout exhibits "Manhattan routing", where wires on neighboring layers are routed orthogonal to each other. This arrangement is a critical VLSI design rule used to minimize the overlap area between adjacent layers, thereby reducing inter-layer capacitive coupling and crosstalk [3]. Fig. 15 offers a closer look at the widths of different layers. Based on the observations, M1 has the thinnest wire width, which is measured to be .06um; M11 has the biggest wire width, which is measured to be .22um. Upper layers are usually wider because they are typically used to distribute power and deliver signals to different part of the chips. These functions require less resistance, therefore larger width, to minimize voltage drop across distance as well as to reduce propagation RC delays.

B. Final Simulation

The post-PNR netlist (including parasitics) was verified using the same testbench. As shown in Fig. 18, the Program Counter increments steadily, and the ALU correctly computes the sum of R0 and R1.

A comprehensive automated check was performed for all ALU operations (Fig. 19). Systematic "found" signals were implemented in the testbench to verify that the result of every operation matches the expected value. The assertion of these signals to logic '1' confirms that the physical design is functionally correct.

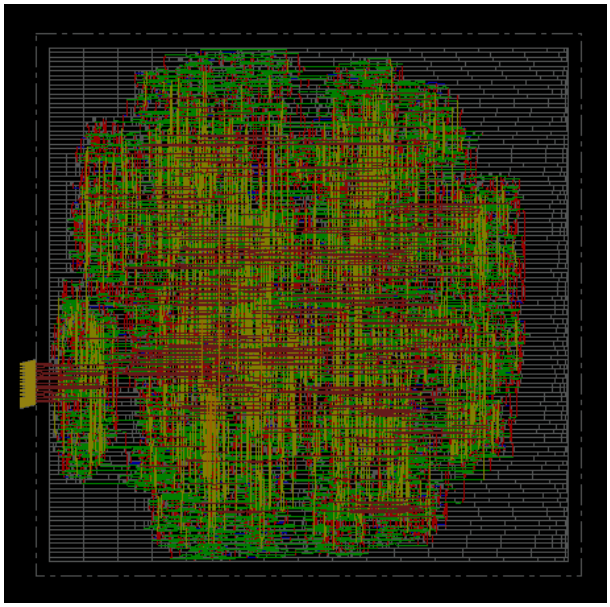


Fig. 11. PNR: Final Layout(signal lines only).

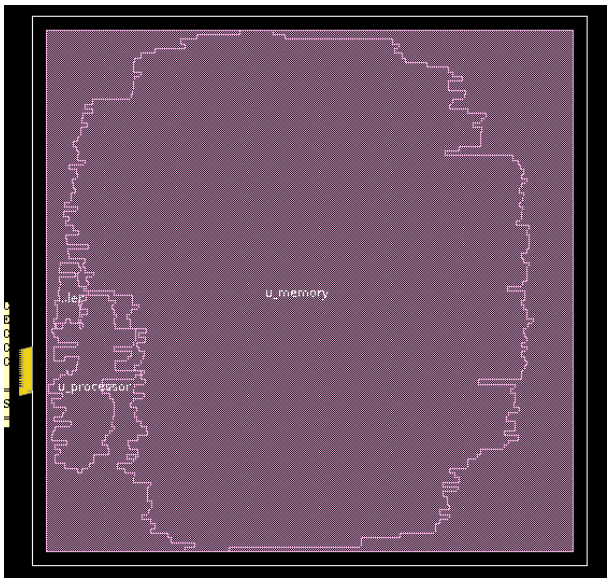


Fig. 12. PNR: Area Allocation.

VI. EFFECT OF FREQUENCY

In the modern era, device performance is heavily dependent on the frequency it can operate on. Therefore, this section examines synthesis and PNR results at several frequencies to better illustrate the relationship between chip physical characteristics and frequency.

A. Frequency Area and Instance Count

Figures 20 and 21 demonstrate that both chip area and instance count increase with frequency. The tighter timing constraints necessitates the insertion of buffers and inverters and replacing the original smaller cells with bigger, high-drive

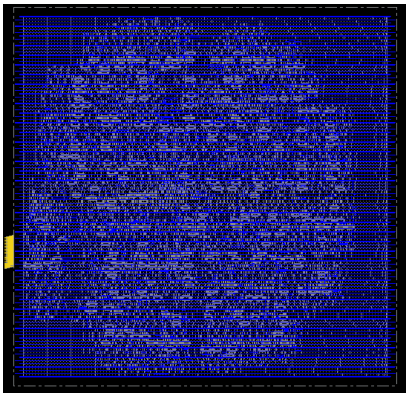


Fig. 13. PNR: Metal layer 1.

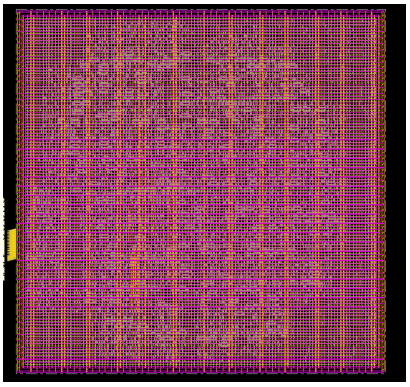


Fig. 14. PNR: Metal layer 6&7 (Orthogonal Routing).

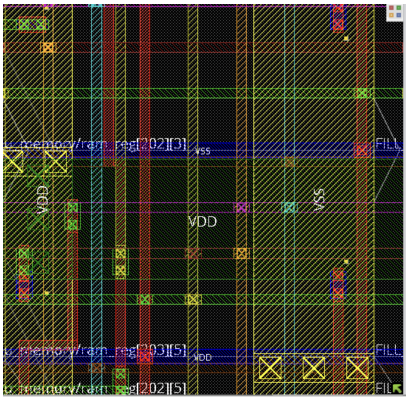


Fig. 15. PNR: Metal layers close-view.

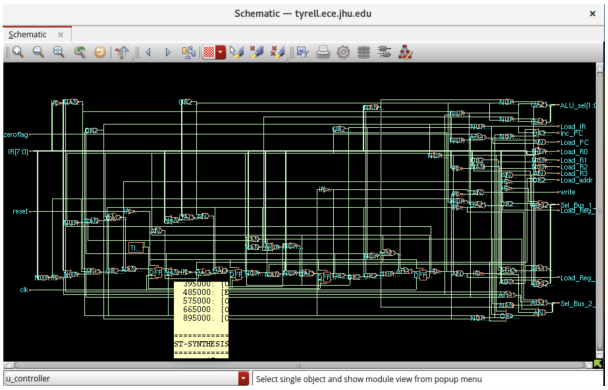


Fig. 16. PNR: Schematics.

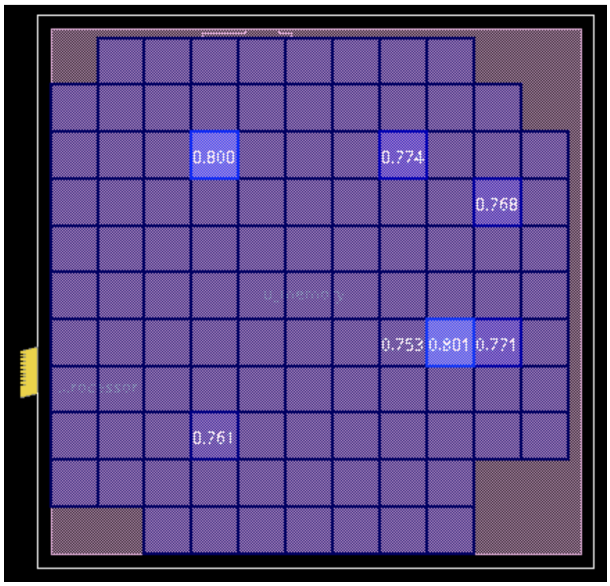


Fig. 17. PNR: Density Map.

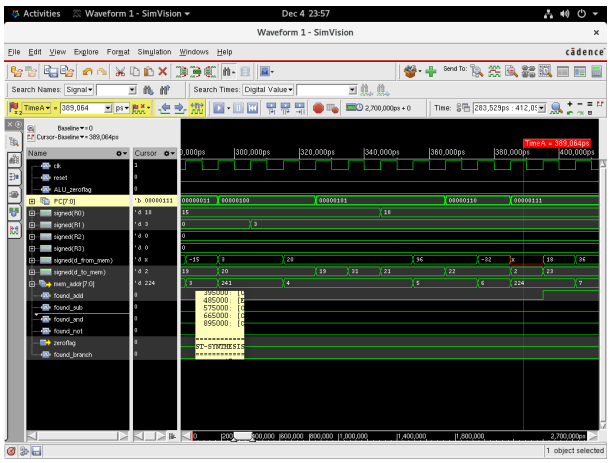


Fig. 18. PNR: ADD simulation.

variants. Generally, PNR area and instance count exceed synthesis estimates due to the addition of physical cells required to resolve realistic interconnect delays. However, a significant exception occurs at 333Hz, where a negative slack is first observed(Fig. 22): the synthesis area unexpectedly surpasses the PNR area. This likely indicates a disagreement in strategies used to meet timing constraints: the synthesis tool may have prioritized aggressive architectural transformations (increasing area) to minimize the slack, while the PNR tool, refrained by physical limitations, performed fewer optimizations while sacrificing final timing.

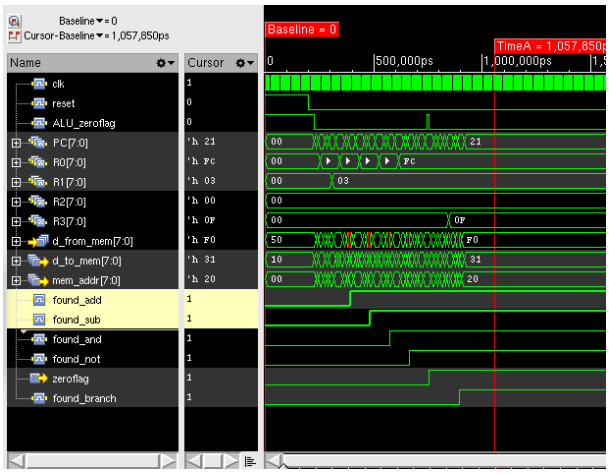


Fig. 19. PNR: Comprehensive operation check.

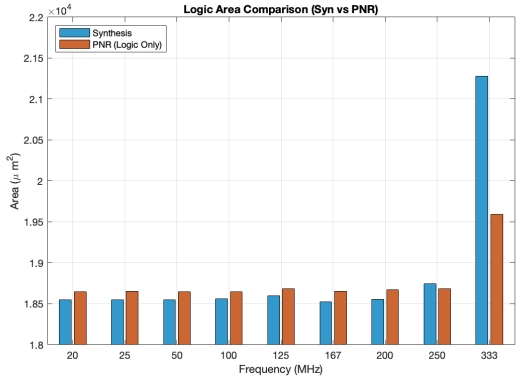


Fig. 20. PNR: Area vs Frequency.

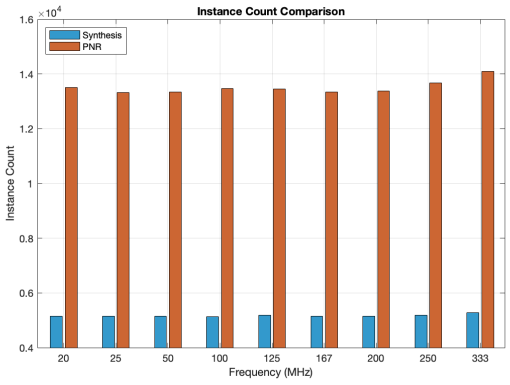


Fig. 21. PNR: Instance Count vs Frequency.

B. Frequency and Internal Power

Dynamic power can be estimated with Eq. 1.

$$P_{\text{dynamic}} = \alpha \cdot C_{\text{load}} \cdot V_{\text{DD}}^2 \cdot f \tag{1}$$

Therefore, in an ideal scenario, a linear relationship is expected between frequency and power if other variables are kept constant. The PNR result clearly illustrates this trend(Fig. 23). Pearson Coefficient is calculated to be .9978 for frequency and internal power after PNR, which confirms a strong linear relationship between the two quantities.

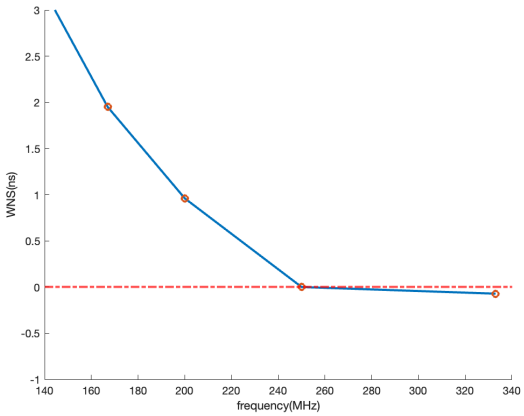


Fig. 22. PNR: WNS vs Frequency.

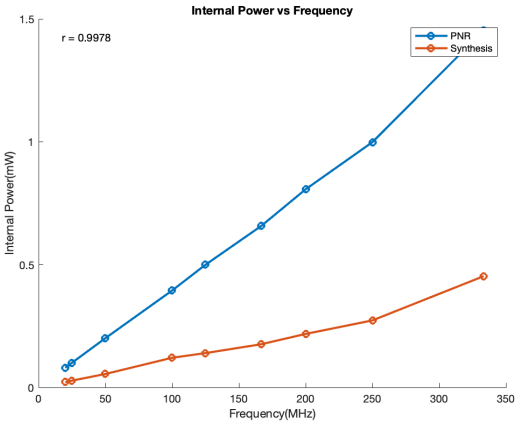


Fig. 23. PNR: Synthesis, PNR Power vs Frequency.

C. Frequency and Leakage Power

The leakage power was hovering around 2.17uW when the slack time was positive. However, there is a sudden explosion when WNS becomes negative, which corresponds to the behavior at 333MHz on Fig. 24. This can be explained by the fact that, to address the negative slack, opimizer tries to replace High-Threshold-Voltage(HVT) cells with Low-Threshold-Voltage(LVT), which effectively increases the performance at high frequency while unavoidably leads to more leakage currents [5]. This phenomenon can be well explained by the subthreshold leakage current equation Eq. 2. As the threshold voltage becomes lower, the exponential term becomes less negative, therefore increases the leakage current and the power.

$$I_{leakage} = \alpha T^2 e^{-\beta \frac{V_{th}}{T}}. \tag{2}$$

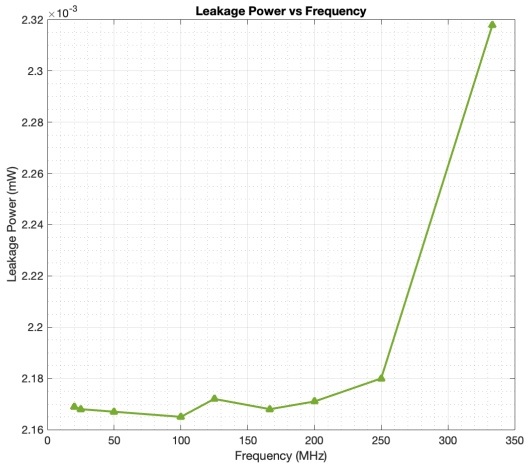


Fig. 24. PNR: Leakage Power vs Frequency.

VII. Conclusion

This paper presented the complete design of an 8-bit RISC processor, walking through a full ASIC workflow from RTL to physical layout. The modular architecture, consisting of a dedicated FSM controller, ALU, and synthesized memory, was successfully modeled in Verilog and verified through test-benches. Simulation results at the behavioral, post-synthesis, and post-layout stages confirmed the functionalities of all instruction types, including arithmetic, logical, and branching operations.

The physical design analysis provided critical insights into the trade-offs between area, power, and performance. It can be concluded that an increase in frequency necessitates an increase in area and power. A negative slack is undesired and leads to severe leakage power. Based on the current design, the best possible frequency it can operate at is 200Hz, which corresponds to a clock period of 5ns.

Acknowledgment

I would like to thank Dr. Tinoosh Mohsenin for her guidance throughout the semester and for making the complex course material engaging. Additionally, I extend my gratitude to our teaching assistant, Utteja, for providing valuable feedback and prompt clarifications during the design process.

References

[1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Morgan Kaufmann, 2013.

[2] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*, 2nd ed. Morgan Kaufmann, 2012.

[3] N. H. E. Weste and D. M. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Addison-Wesley, 2011, ch. 4.

[4] J. M. Rabaey, A. Chandrakasan, and B. Nikolic, *Digital Integrated Circuits: A Design Perspective*, 2nd ed. Pearson, 2003.

[5] N. P. S., "Efficient Leakage Power Reduction in VLSI Circuits using Multi-Threshold Voltage Techniques," *International Journal of Innovative Research in Science, Engineering and Technology (IJIRSET)*, vol. 13, no. 5, May 2024.